

Thread

¿Qué son los hilos?

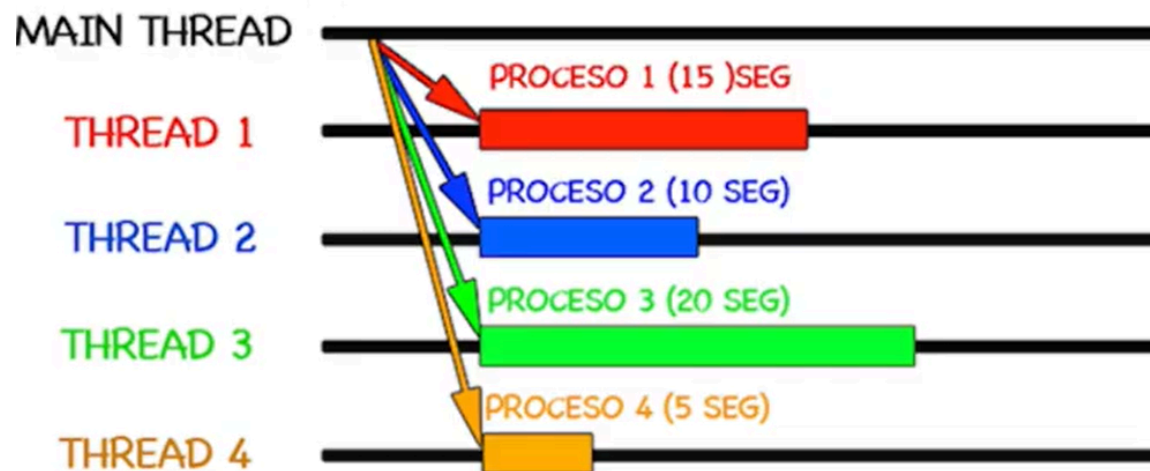
Un hilo es la unidad básica de ejecución más pequeña dentro de un proceso de software, es decir, es lo que hace que tu programa se ejecute y funcione.

Cuando tenemos un programa básico este puede correr sin problemas por un único hilo, el cual se llama hilo principal (main thread), a esos programas se le llaman programas monohilos.

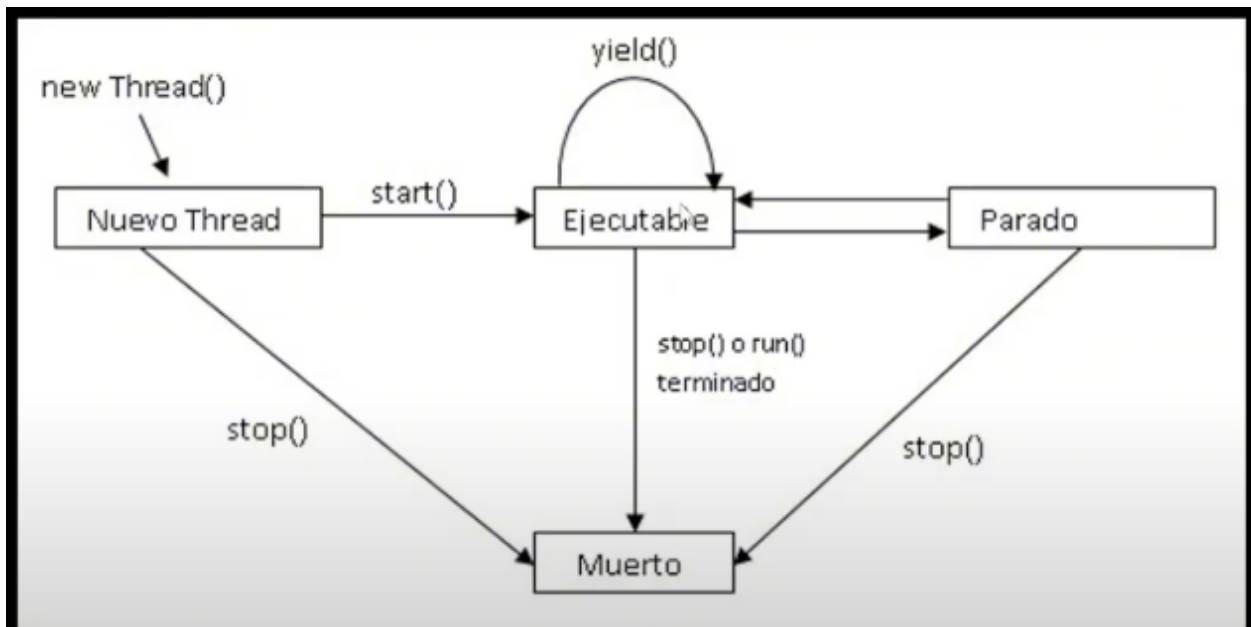
El problema reside cuando el programa que tenemos es más sofisticado y complejo, pedirle al main thread que sea él, el que realice todas las tareas, haría que nuestro programa se vuelva demasiado lento. Imagina que tu programa tiene que descargar archivos de internet y a la vez responder a los clics desde la interfaz de usuario, el hilo principal tiene demasiadas responsabilidades por lo tanto hará que se congele al no poder trabajar simultáneamente.

Para esto sirven los hilos, para separar las responsabilidades. Los hilos nacen del hilo principal, estos permiten la ejecución de varias partes de un programa al mismo tiempo (simultáneamente) en forma de subprocesos sin que uno interfiera con otro y pudiendo ser su duración diferente unos de otros.

A cada hilo se le puede asignar la prioridad con la que queremos que este se ejecute.



Los hilos tienen un ciclo de vida, que pueden seguir alguno de los siguientes órdenes:



Dependiendo de la situación/estado en el que se encuentre, el hilo seguirá un proceso u otro

Métodos:

Algunos de los métodos más importantes que tiene la clase thread son:

- Ejecución:
 - start(): se encarga de que el hilo comience a ejecutarse.
 - run(): es donde se le dan las instrucciones que debe de seguir el hilo (A este método nunca se le debe llamar desde el main, en su lugar llamaremos a start(), que es el que se encargará de llamar a run())
- Control de tiempo:
 - sleep(): se encarga de que la ejecución del hilo sea pausada por unos milisegundos o nanosegundos
 - join(): Se usa cuando necesitas antes el resultado de un hilo para poder ejecutar el hilo. Ejemplo: tienes un Hilo A y un Hilo B. Si el Hilo A necesita que el Hilo B termine su cálculo para poder usar el resultado, el Hilo A utiliza este método para esperar a que el Hilo B termine.
- Control de flujo
 - interrupt()/interrupted(): interrupt() Interrumpe el hilo, sirve para enviar una señal desde fuera a un hilo específico para avisarle de que debería de detenerse o cambiar su comportamiento. En cambio interrupted() sirve para que el propio hilo

compruebe si alguien le ha mandado una señal para que cambie su comportamiento

- `isAlive()`: Comprueba si el hilo está vivo, devuelve un booleano. Es muy útil desde fuera para saber si el hilo sigue vivo o no

¿Cómo crear hilos?

Se pueden crear de dos formas:

- Heredando desde la clase `Thread`:

```
```Java
```

```
public class Corredor1 extends Thread {

 public void run () {

 system.out.println("Hola mundo ")

 }

}
```

```
```
```

- Implementando la interfaz `Runnable`:

```
```Java
```

```
public class TareaCorredor implements Runnable {

 public void run () {

 system.out.println("Hola mundo ")

 }

}
```

```
}
...
```

Al heredar desde la clase `Thread` conseguimos que la clase adquiera todos los comportamientos de y herramientas de un hilo predefinido del sistema.

El problema que se presenta aquí es que Java no tiene herencia múltiple (no deja heredar de varias clases padres a la vez), por lo tanto ya no podríamos heredar de ninguna otra clase. Para solucionar este problema usamos la interfaz `Runnable`.

Con la interfaz conseguimos solucionar este problema pero nos vemos obligados a sobrescribir si o si el método `run`.

En la opción A. Tu clase es el hilo

En la opción B. Tu clase solo es un trabajador que acepta un contrato a cambio de tener que reescribir todos los métodos obligatoriamente

Ejemplo de cómo instanciarlos en el main

```
```Java
```

```
public class PistaDeCarreras {  
    public static void main(String[] args) {  
        System.out.println("¡COMIENZA LA CARRERA!");  
  
        // --- Preparando al corredor 1 (Método Thread) ---  
        CorredorThread corredor1 = new CorredorThread("Atleta Rojo (Thread)");  
        //En esta situación lo que hacemos es decirle al objeto corredor1 que se comporte como  
        //lo que es, ya que el ya es un hilo  
  
        // --- Preparando al corredor 2 (Método Runnable) ---  
        TareaCorredor tarea = new TareaCorredor("Atleta Azul (Runnable)");  
        Thread corredor2 = new Thread(tarea); // Le pasamos la tarea al hilo  
  
        // --- Damos la salida ---  
        corredor1.start();  
        corredor2.start();  
  
        System.out.println("El Hilo Principal ha dado la salida y puede hacer otras cosas...");  
    }  
}
```

```
....
```

Problemas más comunes al trabajar con hilos.

Los hilos son una parte importante de nuestro programa para mejorar la fluidez del mismo, pero a su vez también son frágiles debido a su forma de trabajo.

Esto nos puede generar algunos problemas bastantes comunes:

- **Race Condition:**

Este es el problema más común y el más difícil de detectar, porque a veces el programa falla y a veces funciona perfectamente.

Ocurre cuando dos o más hilos intentan leer y modificar la misma variable exacta al mismo tiempo.

Una buena analogía para esto es la del cajero automático: imagina una cuenta bancaria compartida con 100€. El Hilo A (tú) y el Hilo B (otra persona) vais a dos cajeros distintos al mismo tiempo a sacar 80€. Ambos hilos leen el saldo simultáneamente: "Hay 100€". Ambos piensan que hay dinero suficiente y permiten la operación. Resultado: el banco os ha dado 160€ de una cuenta que solo tenía 100€, dejando un saldo negativo corrupto, porque los hilos se han "pisado" la información.

- **Deadlocks:**

Para solucionar las condiciones de carrera empezamos a poner "candados", en Java por defecto todos los objetos que creemos llevan por defecto un "candado" invisible incorporado.

Por defecto a estos candado nadie le hace caso. Los hilos pueden entrar y leer datos o modificarlos todos a la vez. Como si en una intersección con el coche todos intentaran entrar a la vez.

En ocasiones hay funciones que son peligrosas debido a que modifican datos importantes, para que nuestro programa no entre en un deadlock, usamos la palabra `synchronized`.

`Synchronized` se coloca en la firma del metodo, algo así:

```
public synchronized void sacarDinero(int cantidad.
```

Funciona tal que así:

1. El hilo 1 entra en la función
2. Empieza ejecutar las instrucciones de la función
3. Si otro hilo llega a la función, primero "llama" a la puerta y pregunta si hay alguien, si el hilo 1 sigue en el proceso, el otro hilo tendrá que esperar a que el hilo 1 termine
4. Cuando el hilo 1 termina el proceso, entra el otro hilo que estaba en cola esperando.

En resumen: un deadlock es un callejón sin salida donde el programa se queda congelado para siempre, aunque no dé ningún error por pantalla.

- El Cuello de Botella de la Sincronización:

Cuando por miedo a sufrir condiciones de carrera comenzamos a colocar synchronized en cada uno de nuestros metodos, nuestro programa se ralentizara debido a que todos los hilos tienen que hacer cola para entrar en una función. Lo importante es saber cuando hay que utilizar synchronized y cuando no

Diferencia entre versiones:

Thread vive desde la versión 1.0Java pero los cambios más importantes llegaron en las versiones 9 y 10, a partir de estas versiones se decidió eliminar los métodos stop(), resume() o suspend() debido a que generaban mas problemas de los que solucionaba, los demás métodos han permanecido iguales o con ligeras modificaciones hasta el día de hoy.